

Web-Security

HTTPS, OWASP

HTTPS

Hypertext Transfer Protocol Secure

HTTP ist grundsätzlich unverschlüsselt

- Jeder Mittelsmann kann daher die Kommunikation mitschneiden und verändern (MITM-Angriffe)
- Um sicherzugehen, dass die Inhalte vom Server stammen und unverändert ausgeliefert werden, muss HTTPS genutzt werden

HTTPS: HTTP over TLS/HTTP over SSL/HTTP Secure

- Baut wie HTTPS auf der Anwendungsschicht auf
- Sämtlicher Inhalt (Request, Reponse) wird verschlüsselt
- IP-Adresse und Port sind weiterhin für Mittelsmänner zu erkennen

TLS/SSL

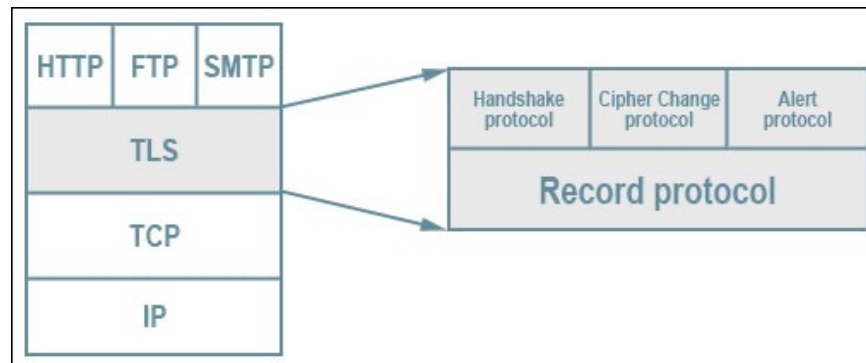
- TLS (Transport Layer Security) ist eine Weiterentwicklung von SSL (Secure Socket Layer) und wird heutzutage fast ausschließlich genutzt
- Die Begriffe *TLS* und *SSL* werden häufig durcheinander gebracht und oft synonym verwendet

HTTPS im RFC 2818 (2000): <https://tools.ietf.org/html/rfc2818>

HTTPS

TLS verschlüsselt die Anwendungsdaten

Schicht zwischen TCP und Application Layer



https://commons.wikimedia.org/wiki/File:TLS_protocol_stack.jpg

Verbindungsaufbau basiert auf asymmetrischer Verschlüsselung

- Hierzu ist ein Public und ein Private Key nötig
- Das Zertifikat liefert hierbei den Public Key, sowie zusätzlichen Meta-Informationen wie dem Servernamen, Gültigkeitszeitraum und Signaturen von anderen Zertifikaten. **Wichtig hierbei ist, dass die Zuordnung zwischen Public Key und dem zugehörigen Subjekt vom Aussteller bestätigt wird, wodurch eine Vertrauensfrage initiiert wird**

Verbindungsaufbau

- Verschlüsselt der Client Daten mit dem Public Key des Servers, so kann er sicher sein, dass auch nur der Besitzer des Public Keys die Daten entschlüsseln kann
- Der Client muss hierfür sicher wissen, dass es sich auch um den korrekten Public Key handelt

Datenaustausch basiert auf symmetrischer Verschlüsselung

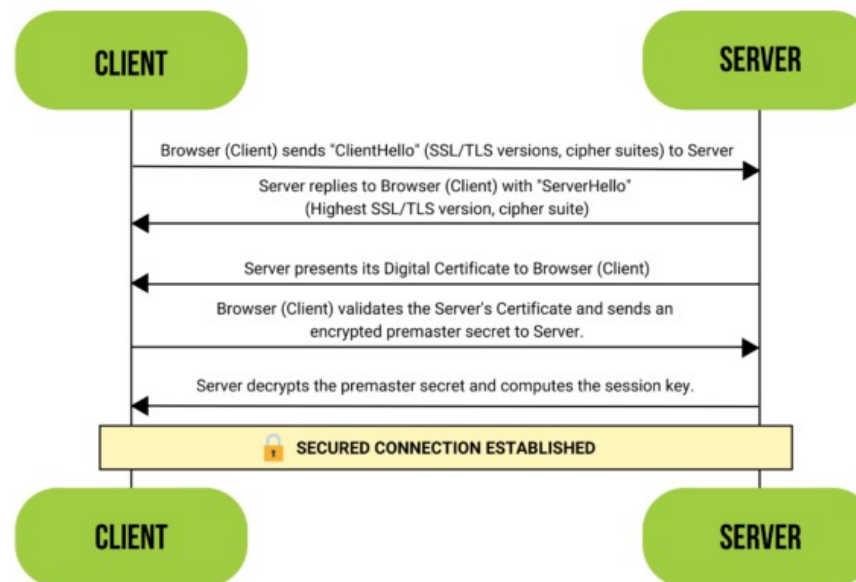
- Client und Server einigen sich auf ein Secret mit dem die Übertragung der Daten passiert (da symmetrische Verschlüsselung schneller ist)

Grundsätzliches Prinzip

- TLS-Handshake (vereinfacht, mit Server-Authentifikation)
 1. Client meldet sich beim Server
 2. Server übermittelt sein Zertifikat inkl. Public Key (und Metadaten wie TLS-Version)
 3. Client prüft das Server-Zertifikat (Public Key) und übermittelt ein Secret
 - Client stellt fest ob das Server-Zertifikat von einem vertrautem CA signiert ist und für die besuchte Webseite ausgestellt wurde (Zertifikatskette)
 - Client nutzt die Metadaten zur Auswahl des zu verwendenden Verschlüsselungsverfahrens
 - Client generiert ein Secret für die Kommunikation
 - Client verschlüsselt das Secret mit dem übermittelten Public Key und sendet es an den Server
 4. Server nutzt seinen Private Key um das Secret zu entschlüsseln und ist somit authentifiziert. Nur Server, die auch den Private Key kennen, werden der folgenden Verschlüsselung auch folgen können
 5. Beide Seiten sind nun im Besitz des Secrets, welches einem symmetrischem Sitzungsschlüssel entspricht; das konkrete Verfahren wurde vereinbart
 6. Die Kommunikation kann von hier an verschlüsselt fortgeführt werden
- Es gibt die Möglichkeit, den Handshake abzukürzen und sich auf einen schon ausgehandelten Sitzungsschlüssel zu beziehen
- Auch kann der Server ein Client-Zertifikat anfordern und dieses mit einer Challenge (Zufallszahl) validieren

Weitere Informationen: <https://www.symantec.com/connect/blogs/how-does-ssl-work-what-ssl-handshake>

SSL/TLS HANDSHAKE



Hinweis: Es wird eine Zertifikatskette bis zum selbstsignierten Zertifikat übertragen



<https://www.ssl.com/article/ssl-tls-handshake-ensuring-secure-online-interactions/>

HTTPS

Handshake kann dauern

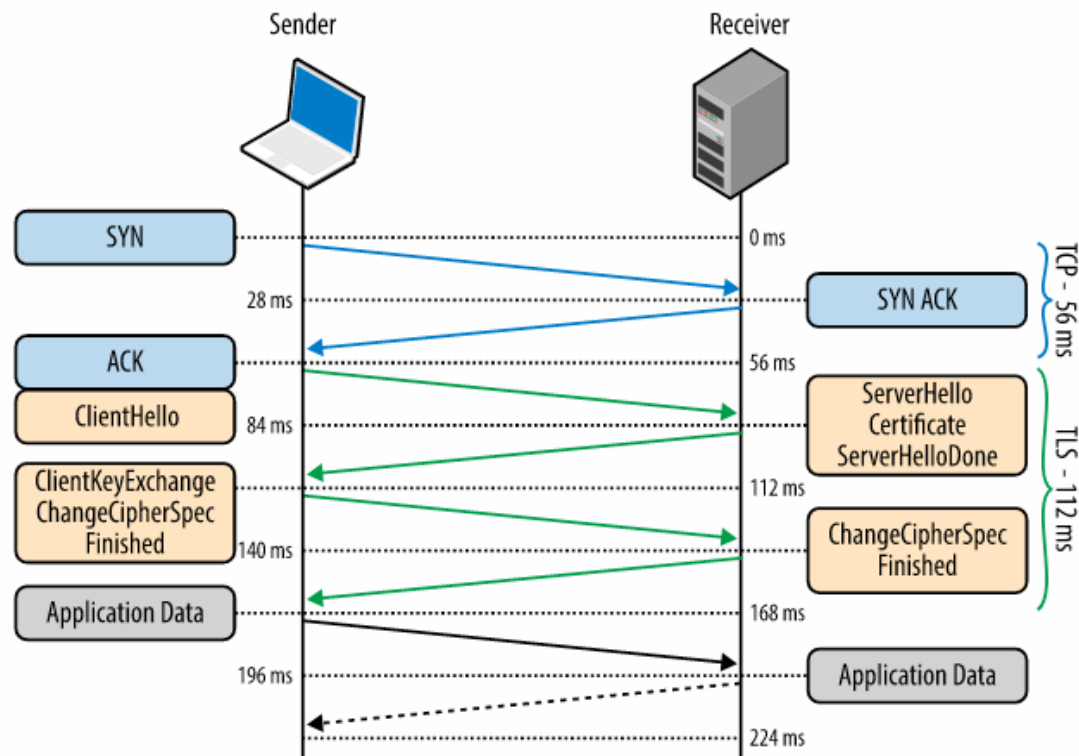
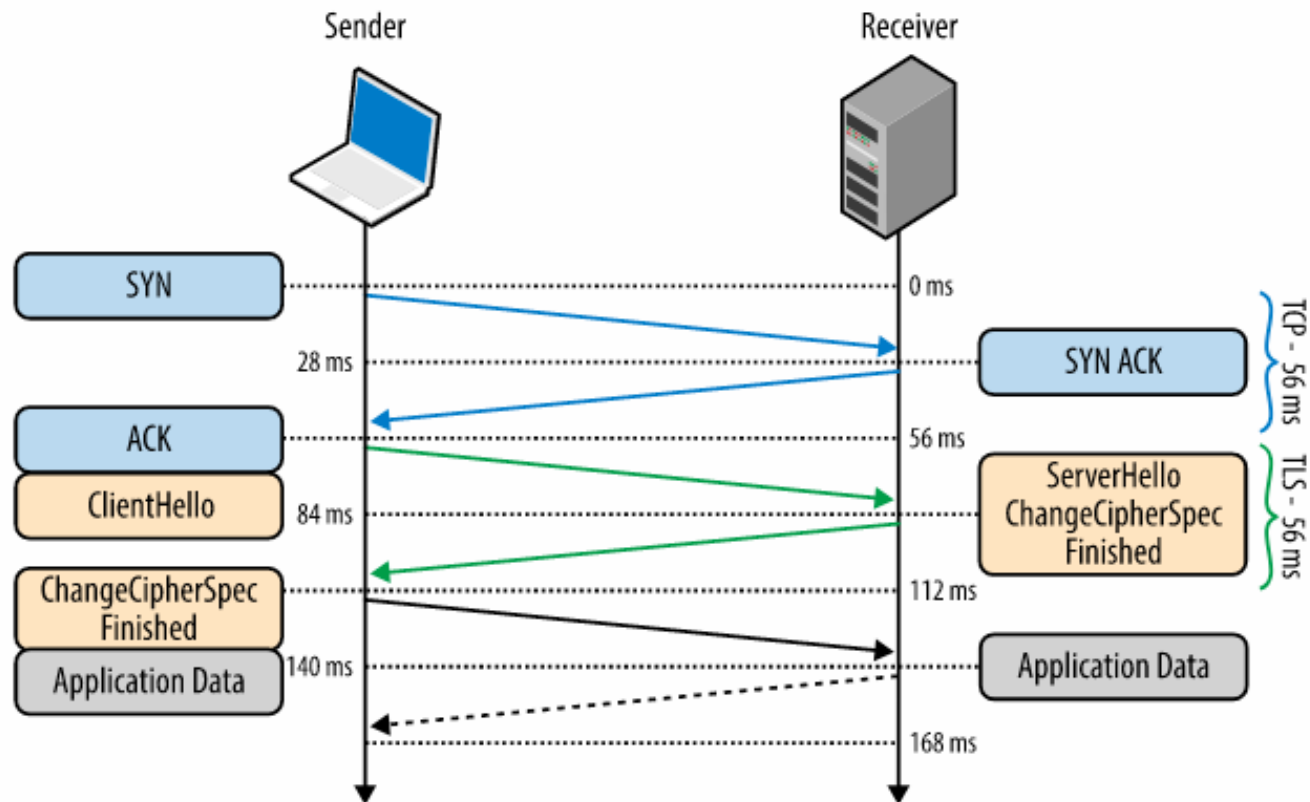


Figure 4-2 assumes the same (optimistic) 28 millisecond one-way "light in fiber" delay between New York and London as used in previous TCP connection establishment examples; see Table 1-1.

<https://istisfastyet.com/>

HTTPS

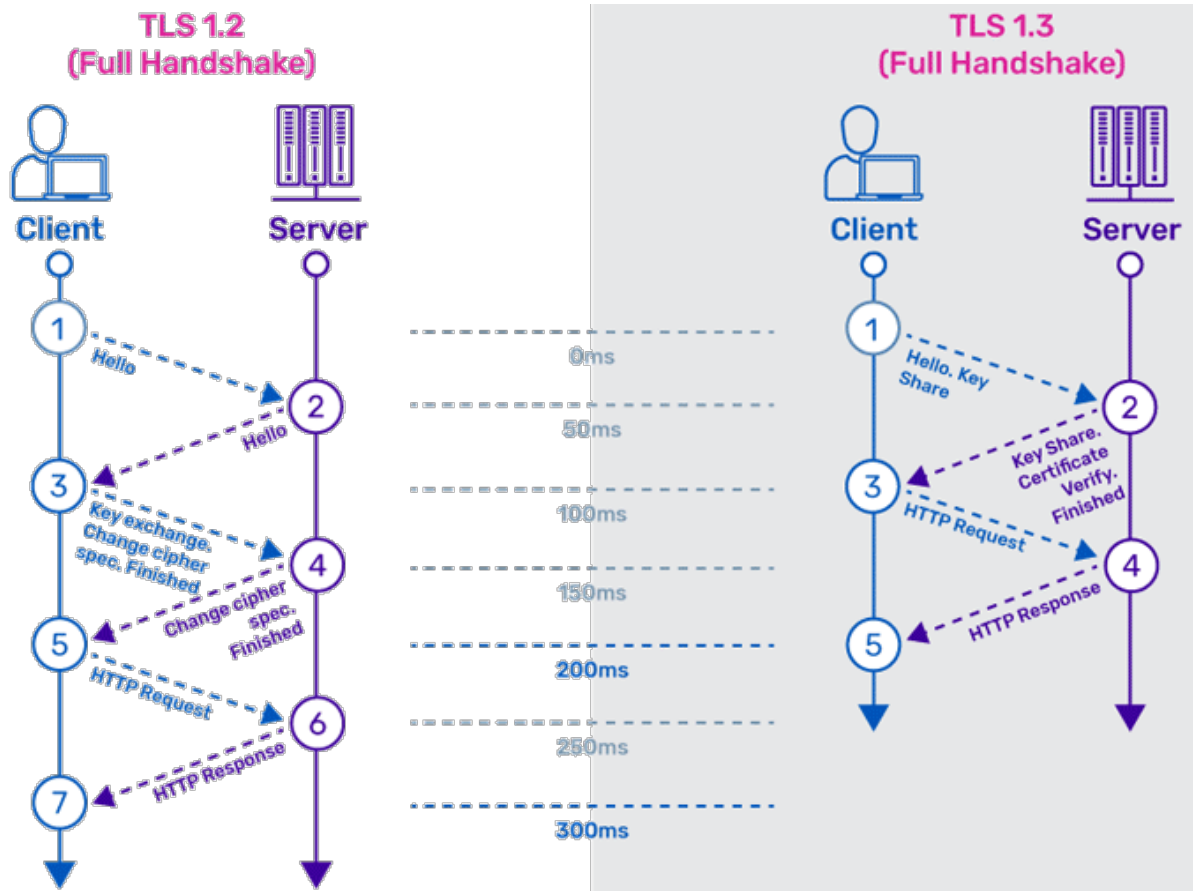
Verkürzter Handshake



<https://istlsfastyet.com/>

HTTPS

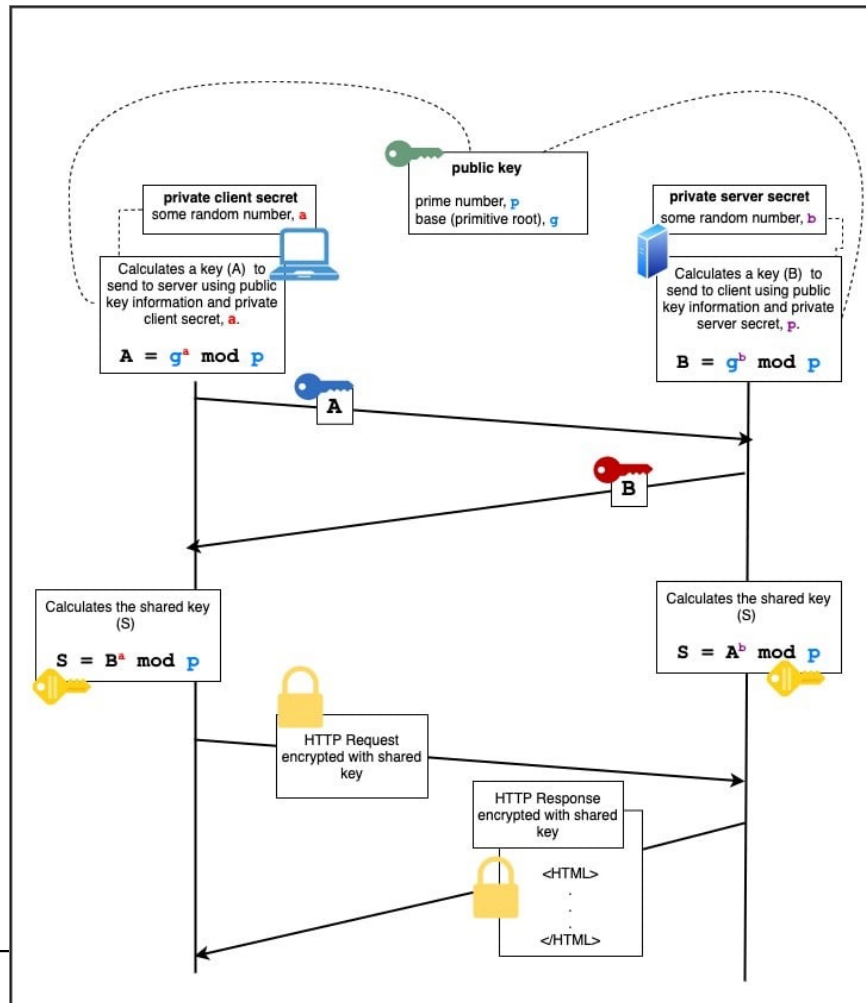
Verbindungsaufbau



<https://www.a10networks.com/glossary/key-differences-between-tls-1-2-and-tls-1-3/>

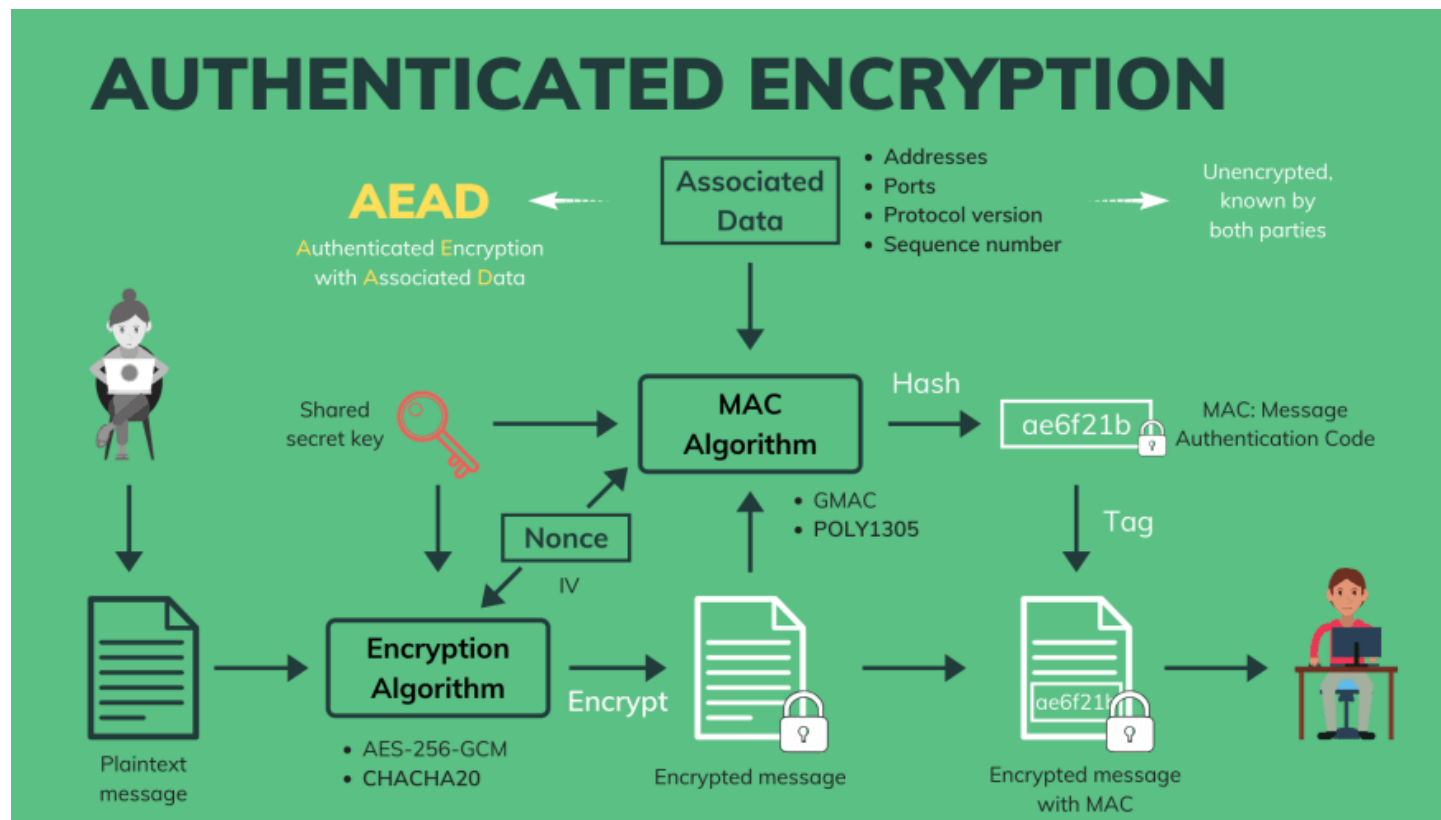
Key Share mittels Diffie Hellmann

<https://thenewstack.io/the-essentials-of-using-an-ephemeral-key-under-tls-1-3/>



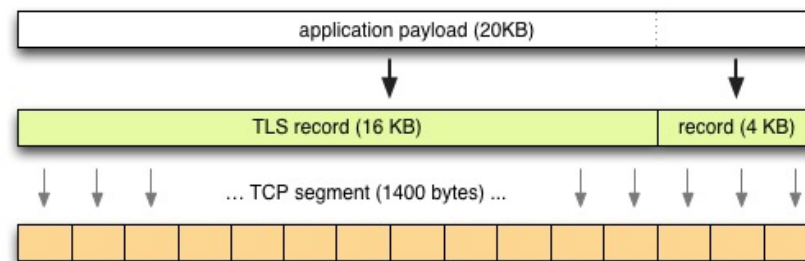
HTTPS Record-Layer

<https://dev.to/techschoolguru/a-complete-overview-of-ssl-tls-and-its-cryptographic-system-36pd>



HTTPS

Record-Layer



Before the application data is encrypted, it is divided into smaller chunks of up to 16KB in size and each of the resulting pieces is signed with a message authentication code (MAC). The resulting data chunk is then encrypted, and the MAC, plus some other protocol metadata forms a “TLS record”, which is then forwarded to the peer:

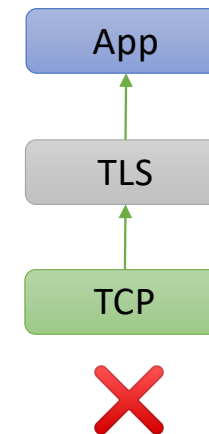
Byte	+0	+1	+2	+3
0	Content type			
1..4	Version		Length	
5..n	Payload			
n..m	MAC			
m..p	Padding (block ciphers only)			

<https://www.igvita.com/2013/10/24/optimizing-tls-record-size-and-buffering-latency/>

HTTP3 - QUIC

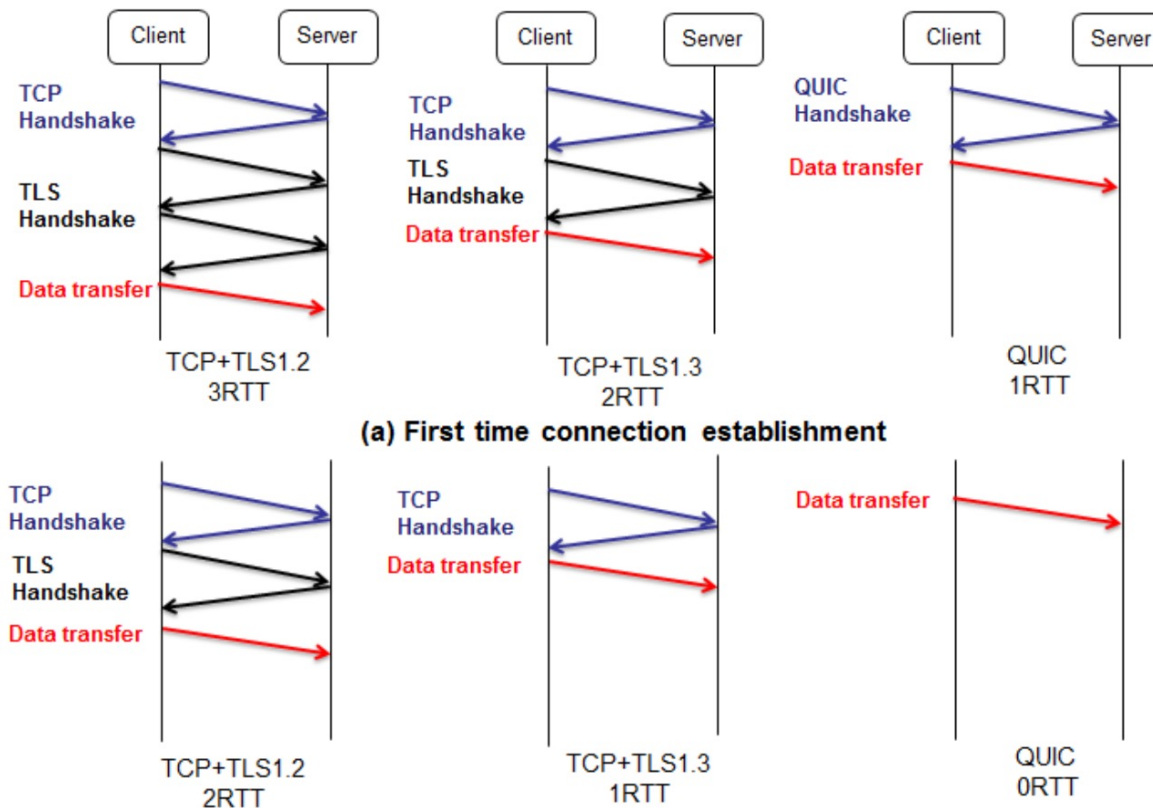
Verkürzter Handshake

- **Optimiert auf geringe Latenz**
 - So wenig RTTs wie möglich
- **Möglichst alles Parallel machen**
 - Verbindungsaufbau
 - Sicherung (über TLS)
 - Applikationsdaten verschicken
- Kosten: Komplexität & Paketgröße



Vergleich der Handshakes

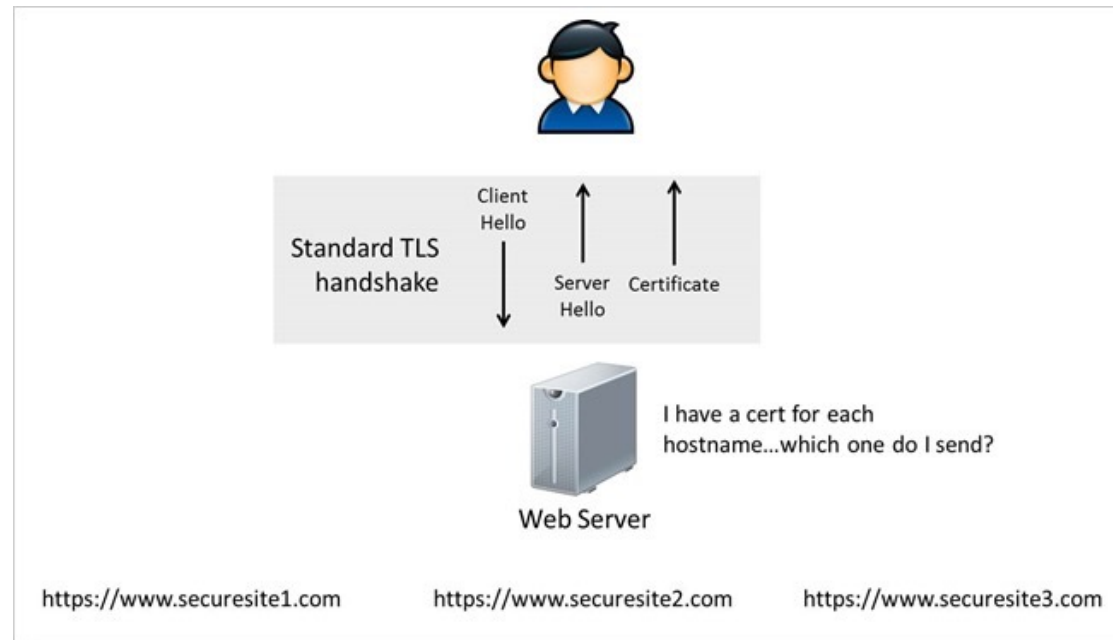
<https://www.research-collection.ethz.ch/bitstream/handle/20.500.11850/129618/QUIC-authors-copy.pdf;jsessionid=634E3E7800CF4C17DA59054A2713C080?sequence=5>



Mit TCP Fast Open ist hier auch ein 0-RTT möglich

TLS und Virtual Hosting

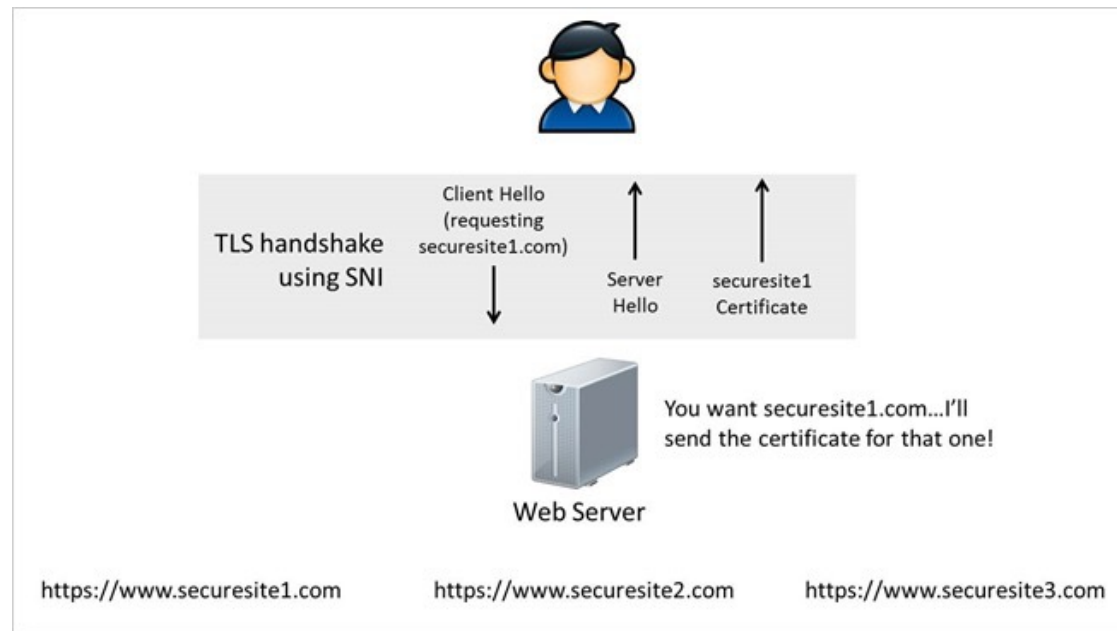
Server Name Indication



<https://devcentral.f5.com/articles/ssl-profiles-part-7-server-name-indication>

TLS und Virtual Hosting

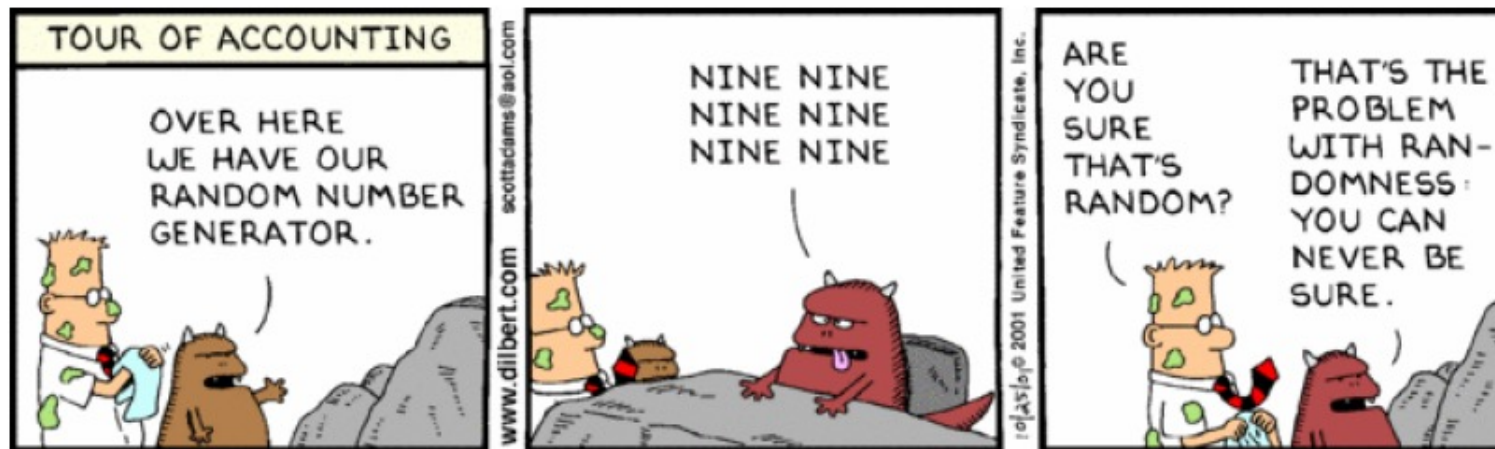
Server Name Indication



<https://devcentral.f5.com/articles/ssl-profiles-part-7-server-name-indication>

TLS

Angriffspunkte?



<http://dilbert.com/strip/2001-10-25>

TLS

Angriffspunkte? Buffer Overflow

Problem:

- Nachlässige Programmierung
- Unsichere Programmiersprache (meist C)
 - ➔ Unzureichende Längenprüfung / Absicherung von Eingabedaten

Angriffstechnik:

- Durch Eingabedaten mit Überlänge (➔ lokale Variablen, Parameter) werden Teile des Stacks überschrieben
- Überschreiben der echten Rücksprungadresse
- Platzieren von eigenem Assemblercode auf dem Stack oder einer gefälschten „Rücksprungadresse“ mit Aufruf einer Bibliotheksprozedur (LoadLibrary, Shell, ..)!

TLS

Angriffspunkte? Buffer Overflow

```
cmd = lies_aus_netz();  
do_something(cmd);  
.....  
int do_something(char* InputString) {  
    char buffer[4];  
    strcpy (buffer, InputString);  
    ...  
    return 0;  
}
```

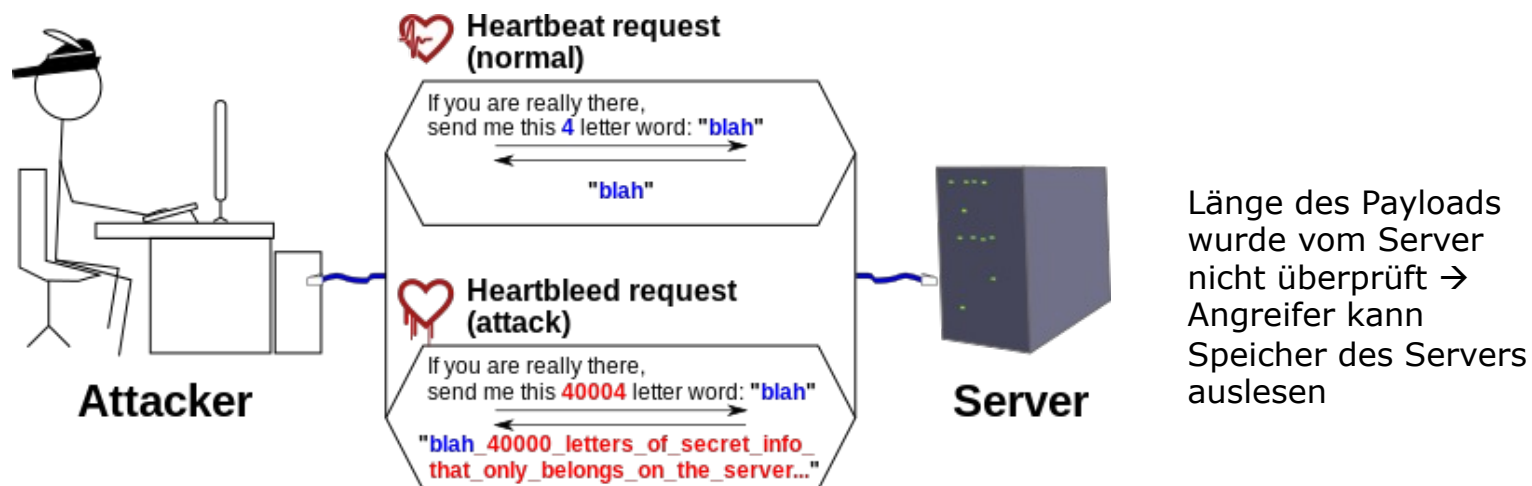
**strcpy kopiert ohne Prüfung solange in
den Speicher, bis NULL gelesen wird!!!**

Zufallszahlengenerator:

- Rechner haben nur einen Pseudozufallszahlengenerator
- Die Entropie der generierten Zahlen ist zu gering
- Von 2004 bis 2013 hatte eine Standardbibliothek zur Erzeugung von Zufallszahlen eine Backdoor für die NSA

Implementierungsfehler: Poodle,

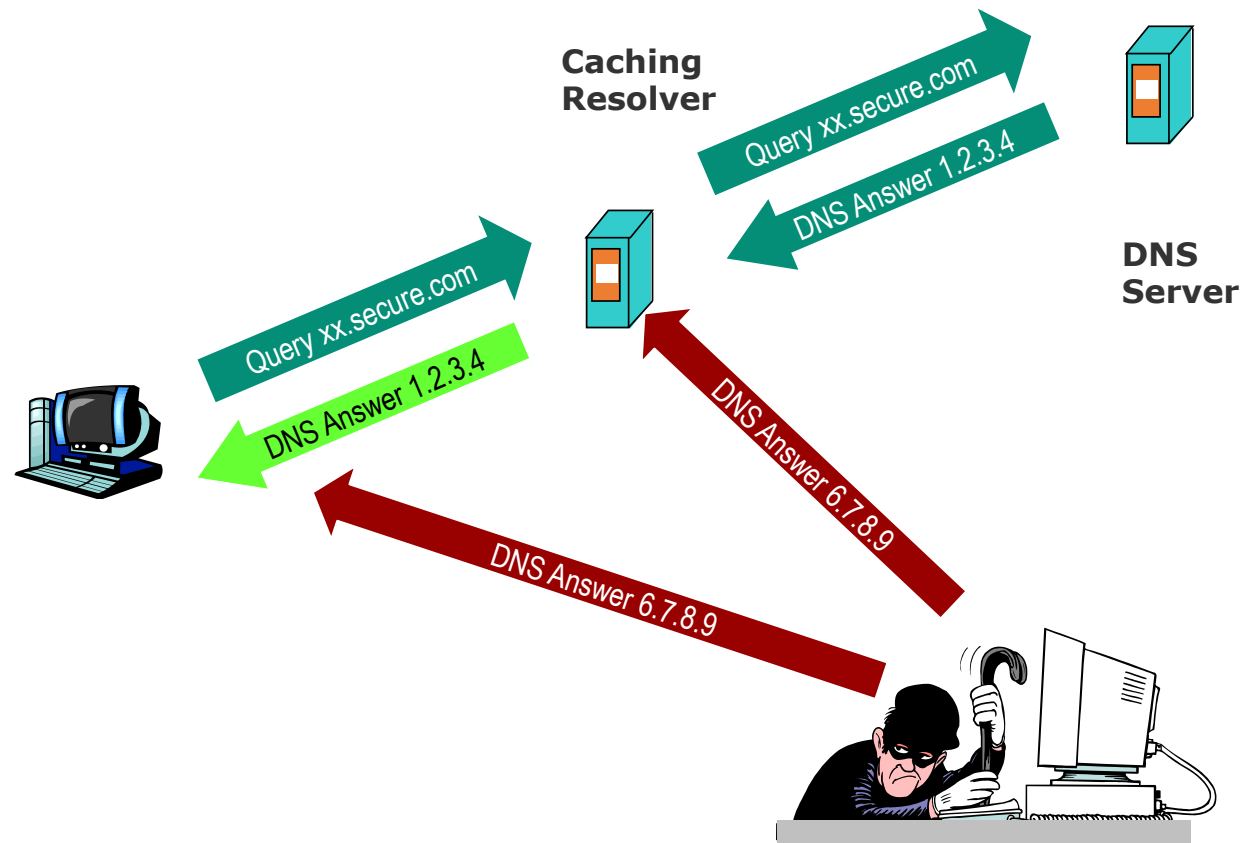
Heartbleed-Bug (OpenSSL):



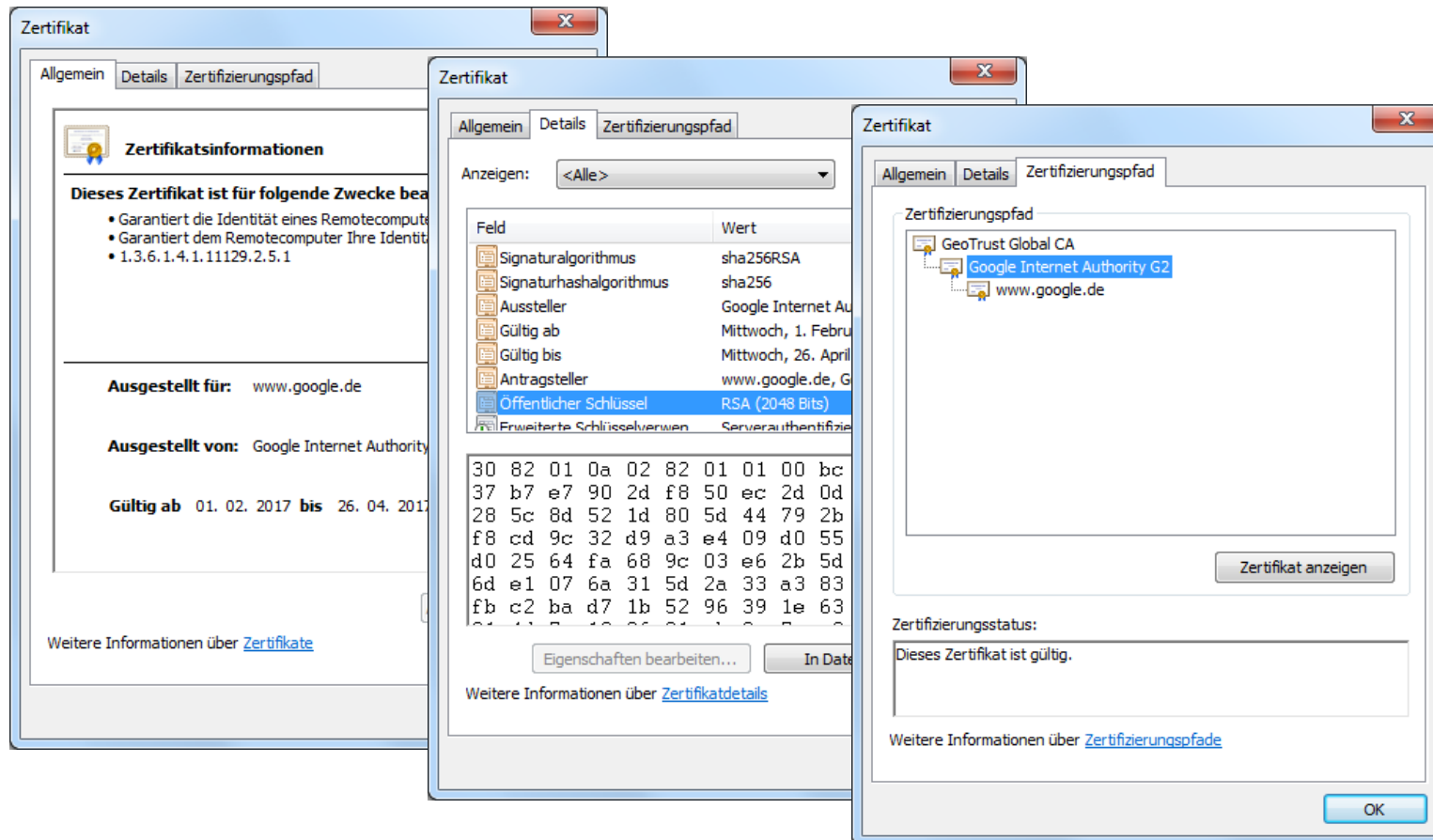
https://upload.wikimedia.org/wikipedia/commons/thumb/c/cb/Heartbleed_bug_explained.svg/1024px-Heartbleed_bug_explained.svg.png

TLS/SSL

Angriffspunkte

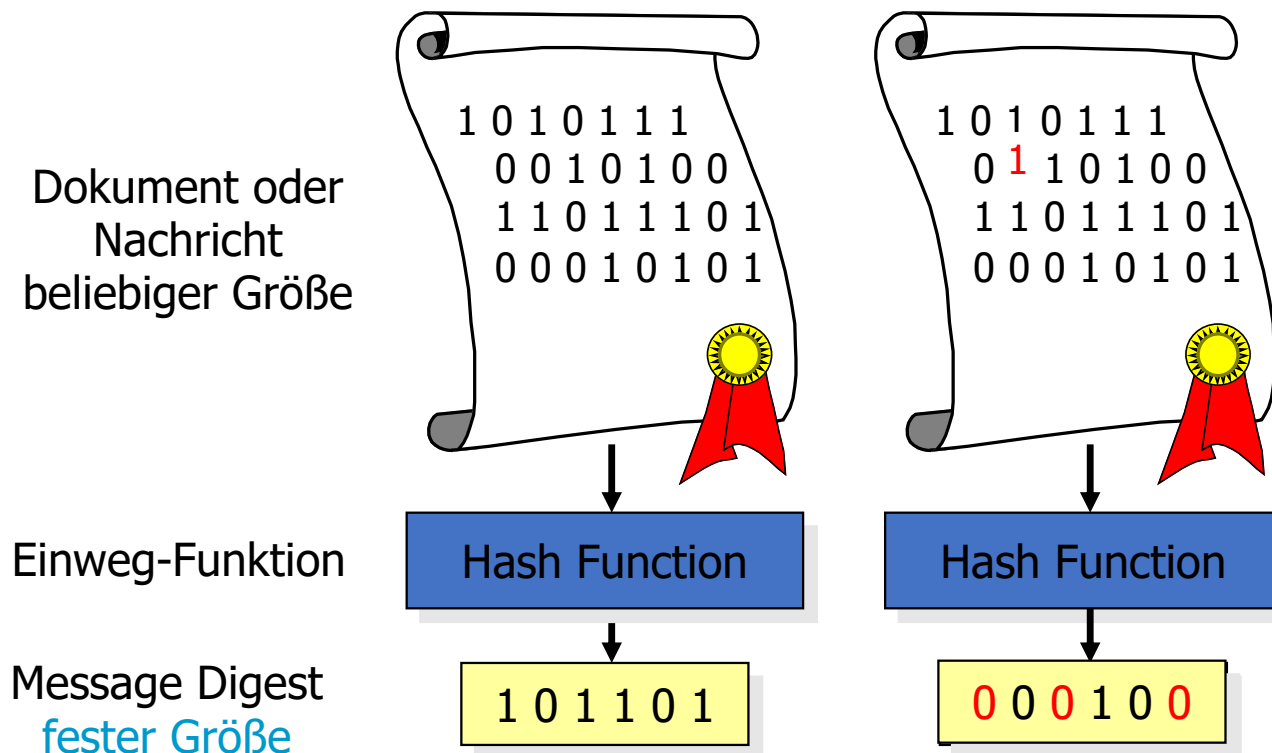


HTTPS Zertifikate



HTTPS

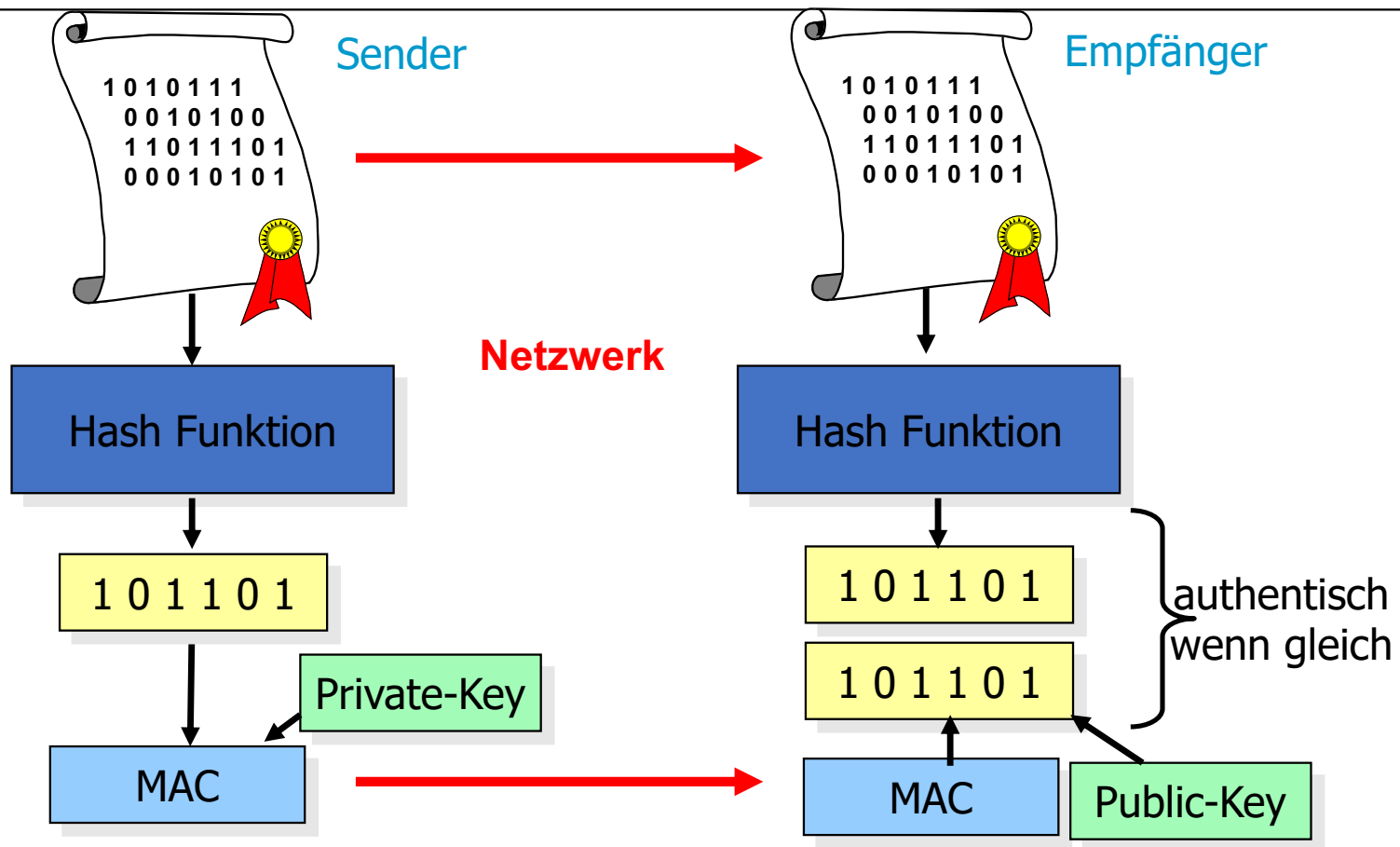
Signaturen



Die Änderung eines einzelnen Bits im Dokument sollte ungefähr 50% der Hash-bits verändern!

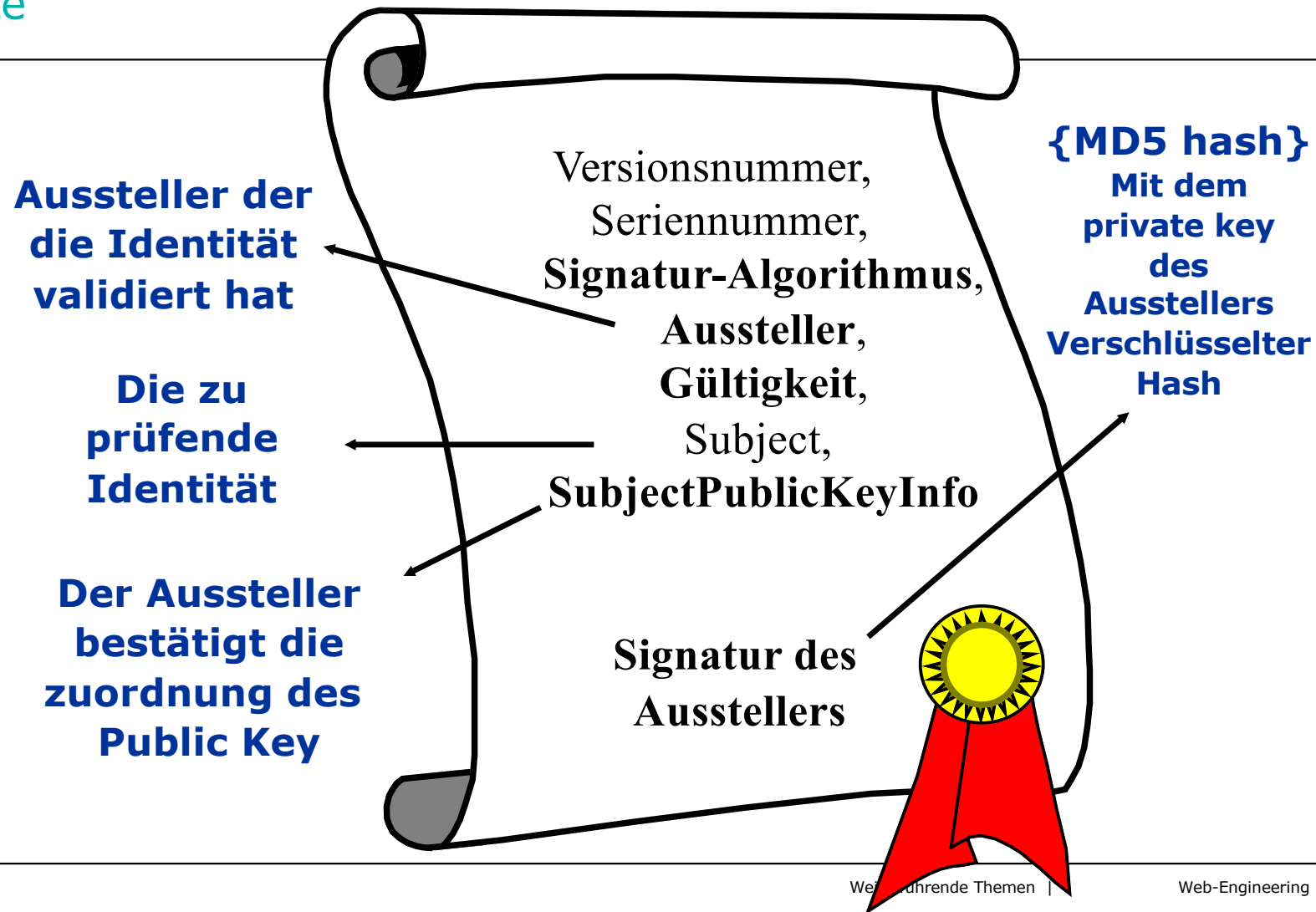
HTTPS

Einsatz von Signaturen



HTTPS

Zertifikate



HTTPS

Certificate Authority (CA)

Vertrauen in Certificate Authorities

- CAs stellen u.a. für Webseitenbetreiber Zertifikate aus und signieren diese (damit benötigen diese auch ein Zertifikat)
- Vertraut der Client der CA, so vertraut er auch dem ausgestellten Zertifikat
- Browser und Betriebssysteme kommen mit einer Reihe voreingestellten „Trusted Root CAs“

Trusted Root CA

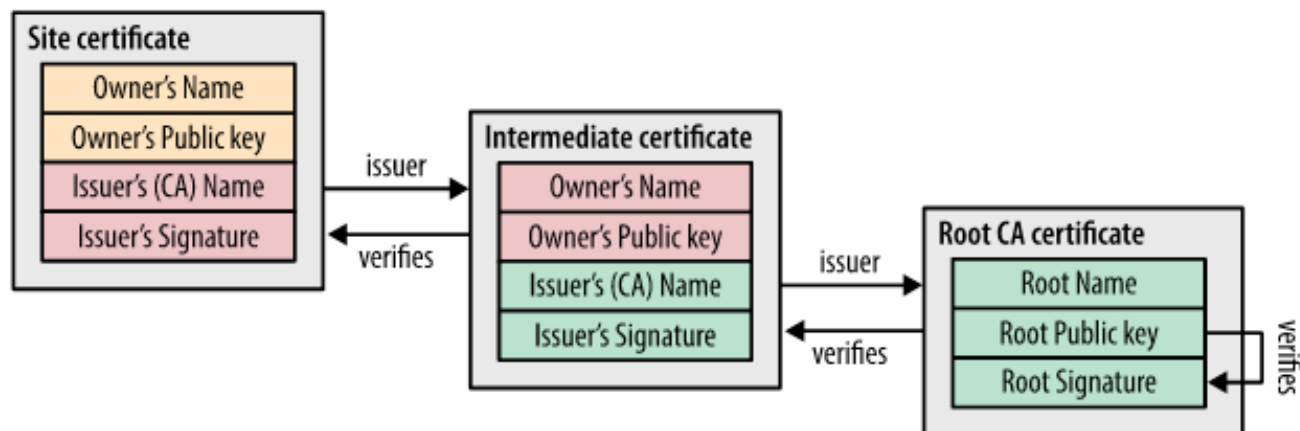
- Kann Zertifikate für weitere CAs ausstellen
 - > Beispiel: GeoTrust CA -> Google CA
- Muss ein längeres Verfahren durchlaufen bis Browserhersteller oder Betriebssysteme die CA aufnehmen;
Selbst-Signiert!

Trusted CA

- Kann beliebige Zertifikate ausstellen
- Muss i.d.R. ein längeres Verfahren durchlaufen bevor sie als CA akzeptiert werden

HTTPS

Vertrauensketten



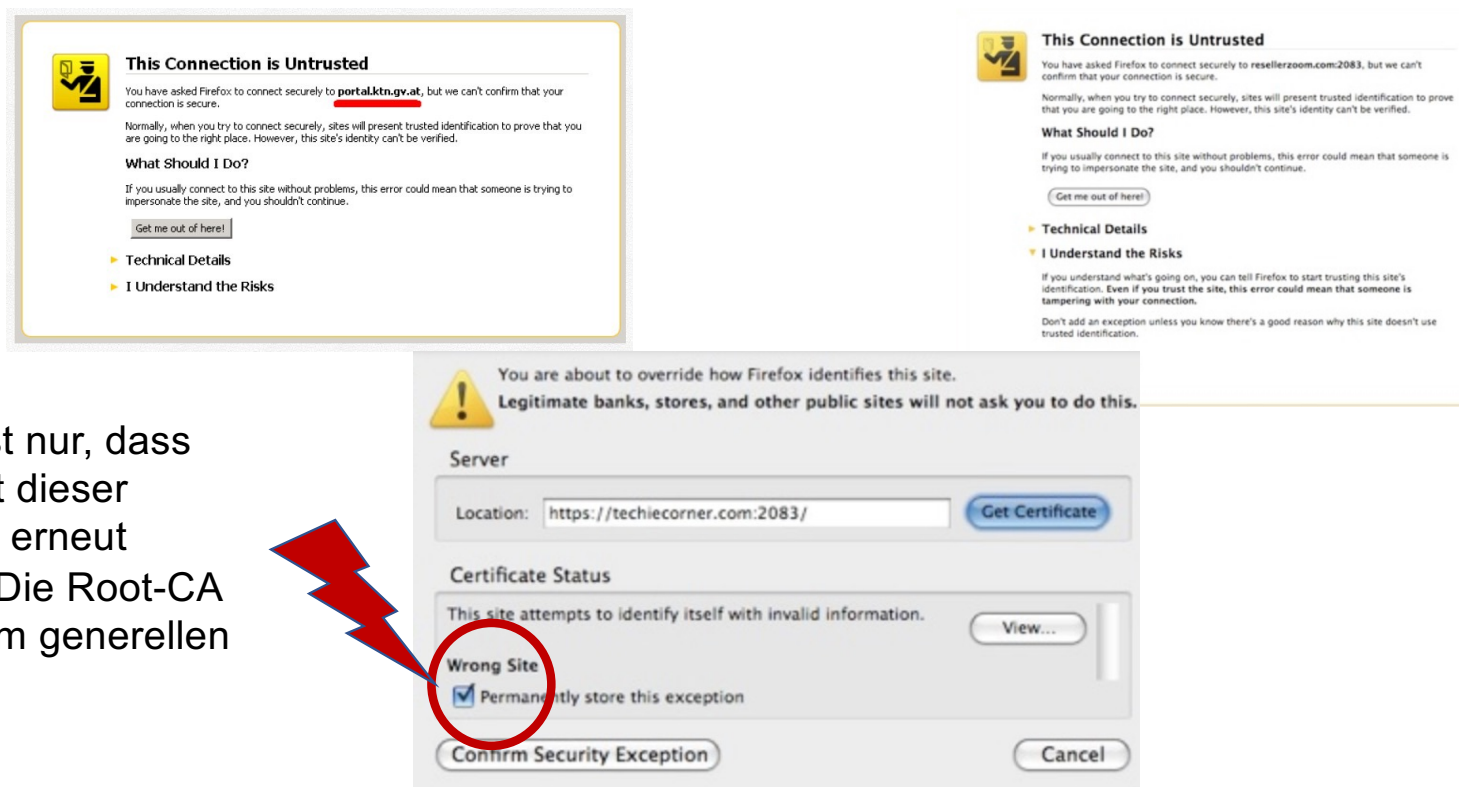
<https://hpbn.co/transport-layer-security-tls/#tls-handshake>

HTTPS

Certificate Authority (CA)

Browser speichern Trusted (Root) CAs

Falls die Web-Seite ein Zertifikat einer bisher nicht aufgelisteten CA liefert, so wird nachgefragt:
Aufpassen!



Permanent heißt nur, dass für das Zertifikat dieser Web-Seite nicht erneut angefragt wird. Die Root-CA selber ist nicht im generellen Trusstore

HTTPS

Certificate Authority (CA)

Anzahl an Certificate Authorities

- Unterschiedliche Anzahl in Betriebssystemen und Browsern vorinstalliert
- Variiert sogar bei der Version

Ein einziger kompromittierter CA gefährdet das gesamte Web

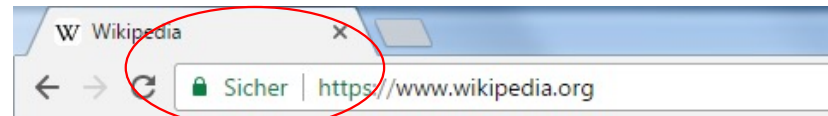
- Zahlreiche Vorfälle in der Vergangenheit zeigen wie anfällig das Verfahren ist
 - > Bekannter Fall: DigiNotar 2011
 - Angreifer kamen in Besitz des Private Keys der CA und stellten sich Zertifikate für über 500 Domains (u.a. google.com) aus, u.a. um iranische Bürger abzuhören
 - In 2011 nutzten über 300.000 Seiten das gefälschte google-Zertifikat (99% im Iran)

Quellen & weitere Informationen: <https://www.eff.org/observatory>, <https://www.eff.org/files/defconssliverse.pdf>, <https://heise.de/-1336603>

Arten von X.509-Zertifikaten

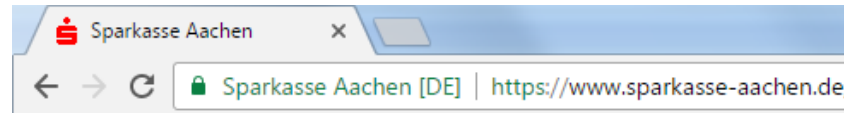
▪ „Normales“ Zertifikat

- > CA garantiert die Echtheit der Webseite (Organization Validated)
 - Prüfverfahren der CA des Antragsstellers der CA ggf. recht lose
- > Kosten: ≥ 0 Euro (z.B. kostenlos über <https://letsencrypt.org/>)



▪ Extended-Validation Zertifikat

- > CA garantiert die Echtheit und den Inhaber der Webseite
 - Feststellung der Identität und der Geschäftsadresse des Antragstellers
 - Sicherstellung, dass der Antragsteller ausschließlicher Eigentümer der Domain ist oder eine exklusive Nutzungsberechtigung hat
 - Sicherstellung, dass die antragstellenden Personen befugt sind und dass rechtlich bindende Dokumente von zeichnungsberechtigten Personen unterschrieben werden
- > Das normale Zertifikat enthält nur die (technische) Aussage, ob die Webseite verschlüsselt ist
- > Kosten: ≥ 200 Euro



However, it all comes down to the human factor. An EV certificate means that the domain is owned by a registered legal entity. It does not mean that site is de facto and de jure trustworthy. In 2016, Troy Hunt **pointed out correctly**, that the effectiveness of EV certificates depends on how people and organizations realize their value and act to protect them. Flash forward a few years later, and he's thinking the days of using **EV certificates are gone**.

Nevertheless, if EV certificates are to be successful it requires **technical controls** for managing their lifecycle that can be enforced without relying on the user.

<https://www.keyfactor.com/blog/what-are-extended-validation-certificates-and-are-they-dead/>

Google and Mozilla have decided to eliminate visual signals in their Chrome and Firefox desktop browsers of special digital certificates meant to assure users that they landed at a legitimate site, not a malicious copycat.

The certificates, dubbed "Extended Validation" (EV) certificates, were a subset of the usual certificates used to encrypt browser-to-server-and-back communications. Unlike run-of-the-mill certificates, EVs can be issued only by a select group of certificate authorities (CAs); to acquire one, a company must go through a complicated process that validates its legal identity as the site owner. They're also more expensive.

<https://www.computerworld.com/article/3431667/chrome-firefox-to-expunge-extended-validation-cert-signals.html>

HTTPS

Aber...

<https://www.troyhunt.com/extended-validation-certificates-are-really-really-dead>

“

Through our own research as well as a survey of prior academic work, the Chrome Security UX team has determined that the EV UI does not protect users as intended. Users do not appear to make secure choices (such as not entering password or credit card information) when the UI is altered or removed, as would be necessary for EV UI to provide meaningful protection.

”

“

The effectiveness of EV has been called into question numerous times over the last few years, there are serious doubts whether users notice the absence of positive security indicators and proof of concepts have been pitting EV against domains for phishing.

”

“

More recently, it has been shown that EV certificates with colliding entity names can be generated by choosing a different jurisdiction. 18 months have passed since then and no changes that address this problem have been identified.

”

How to Get a VeriSign Certificate

- Pay VeriSign (\$300)
- Get DBA license from city hall (\$20)
 - No on-line check for name conflicts; **can I do business as Microsoft?**
- Letterhead from company (free)
- Notarize document (need driver's license) (free)
- **Easy to get fraudulent certificate**
 - Maybe hard to avoid being prosecuted afterwards...
- But this is just VeriSign's policy
 - **many** other CAs...

<http://www0.cs.ucl.ac.uk/staff/B.Karp/gz03/f2016/lectures>

Public Key Infrastructure

Exkurs (Früher gab es hier viele Probleme)

In mid-March 2001, VeriSign, Inc., advised Microsoft that on January 29 and 30, 2001, it issued two VeriSign Class 3 code-signing digital certificates to an individual who fraudulently claimed to be a Microsoft employee. The common name assigned to both certificates is "Microsoft Corporation".

<https://docs.microsoft.com/en-us/security-updates/securitybulletins/2001/ms01-017>

Public Key Infrastructure

Die Probleme sollten gelöst sein

- **Certificate Transparency (CT):** Öffentliche TLS-Zertifikate sind in Logs einsehbar; Missausstellungen werden schneller entdeckt
- **CAA DNS-Records:** Domaininhaber kann festlegen, welche CAs überhaupt Zertifikate ausstellen dürfen
- Browser/Root-Programme sanktionieren CAs bei Verstößen (bis hin zu Distrust)
- **Strengere Branchenregeln und Audits** (WebTrust/ETSI, CA/B-Forum-Regeln; für Code Signing gibt es zusätzliche Anforderungen).
- **Bessere Nachweisführung** für Organisationen und autorisierte Antragsteller (wer darf für eine Firma Zertifikate bestellen).

Public Key Infrastructure

Exkurs

Stuxnet (enthielt Zertifikate für JMicon & Realtek)

Verwendet gestohlene Zertifikate

Entdeckt von **ESET**



Stuxnet (2010) wurde u. a. mit **gültigen Code-Signing-Zertifikaten** signiert, die auf **Realtek** und später **JMicon** ausgestellt waren (beides taiwanische Hardwarehersteller)

Die Angreifer hatten dabei nicht „nur“ ein Zertifikat, sondern den **privaten Schlüssel** dazu (Key theft). Dadurch konnten sie Treiber signieren, die für Windows wie legitime Hersteller-Software aussahen

Der Einsatz von Zertifikaten erfordert

- Das Wissen um die Prozesse zur Erstellung von Zertifikation
 - > Wie wird die Identität sicher gestellt
 - > Wird alles von einer Organisation durchgeführt
- Die Pflege von Zertifikatsperrlisten (certificate revocation lists)
 - > Zertifikate können ihren Zweck verlieren, z.B. bei Verlust des Private Keys
 - > Listen mit zurückgezogenen, abgelaufenen und für ungültig erklärten Zertifikaten

Organisation, die einen PKI-Dienst anbietet: **Trust-Center**

Gesetzliche Vorschriften

- Signaturgesetz von 1997
- EU-Richtlinie
- Signaturgesetz (SigG) von 2000
- Signaturverordnung
- Gesetz zur Anpassung der Formvorschriften (2001)

Public Key Infrastructure (PKI)

Bestandteile

Registrierungsstelle (Registration Authority, RA):

- Organisation, bei der Personen, Maschinen oder auch untergeordnete Zertifizierungsstellen Zertifikate beantragen können
- Diese prüft die Richtigkeit der Daten im gewünschten Zertifikat und genehmigt den Zertifikatsantrag, der dann durch die Zertifizierungsstelle signiert wird

Zertifizierungsstelle (Certificate Authority, CA):

- Vertrauenswürdige Organisation, die die Signatur von Zertifikatsanträgen gemäss bekannter Regeln übernimmt

Verzeichnisdienst:

- ein durchsuchbares Verzeichnis, welches ausgestellte Zertifikate enthält

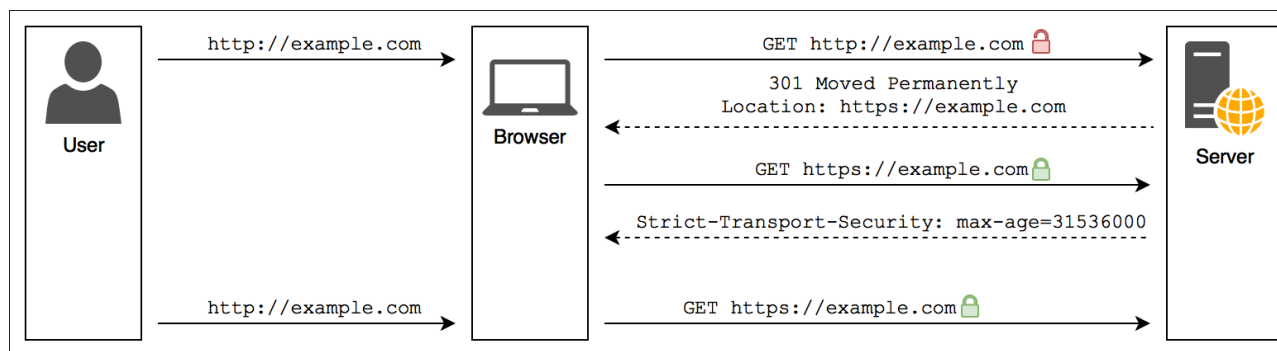
Validierungsdienst:

- Ein Dienst, der die Überprüfung von Zertifikaten in Echtzeit ermöglicht

Strict Transport Security (HSTS)

Problem: Ausschalten oder Downgrading der TLS-Verschlüsselung mittels redirect möglich

- POODLE-Angriffe: [Padding Oracle](#) On Downgraded Legacy Encryption
 - > 2014 konnte mittels eines Downgrade Dance die Verbindung auf SSL Version 3.0 (pre TLS) reduziert werden
 - > Man-in-the-Middle-Angriffe um darauf hin Klartext zur Kommunikation mit dem echten Server zu erhalten
- Durch Verwenden des HSTS-Headers wird dem Webbrowser mitgeteilt, dass eine Seite nur über https und den angegebenen Cipher Suites erreichbar ist. Die Erreichbarkeit über HTTPS merkt er sich für eine bestimmte Dauer (max-age). Künftige HTTP-Anfragen werden automatisch zu HTTPS umgeleitet, ohne Zwischenschritte und ohne Spielraum für Angriffe.



HTTP

Strict Transport Security (HSTS)

```
const hsts = require('hsts')

app.use(hsts({
  maxAge: 15552000 // 180 days in seconds
}))
// Strict-Transport-Security: max-age: 15552000; includeSubDomains
```

<https://www.npmjs.com/package/hsts>

Dem Browser wird über den Strict-Transport-Security -Header mitgeteilt, dass er für die nächsten 180 Tage https benutzen soll

```
// Make sure you run "npm install helmet" to get the Helmet package.
const helmet = require('helmet')

// Sets "Strict-Transport-Security: max-age=5184000; includeSubDomains".
const sixtyDaysInSeconds = 5184000
app.use(helmet.hsts({
  maxAge: sixtyDaysInSeconds
}))
```

“Helmet” ist eine Ansammlung von neun kleineren Middleware-Angeboten, über die sicherheitsrelevante HTTP-Header (u.a. hsts) festgelegt werden

HTTP

Absichern der Zertifikate

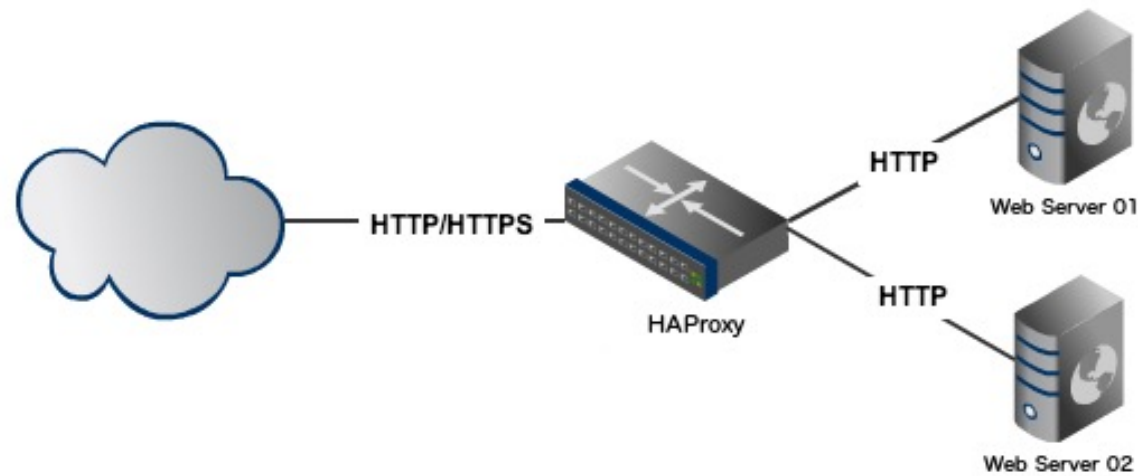
- Früher wurde ein Pinning der Zertifikate unterstützt
- HTTP Public Key Pinning (HPKP) sicherte Zertifikate über einen im Browser gespeicherten Hash ab
- HPKP hat aber durchaus Risiken und konnte auch für Attacken genutzt werden
- Die meisten Browser unterstützen das nicht

Certificate Transparency (CT)

- Offenes Framework um sich gegen den Missbrauch von Zertifikaten zu schützen
- Antwort auf den Vertrauensverlust durch das Hacken von DigiNotar (Root-CA in 2011)
- RFC9162 standardisiert ein öffentliches Log zur Protokollierung aller ausgestellten Zertifikate von CAs
- Firefox testet dies nicht
- https://ssllmate.com/labs/ct_policy_analyzer/

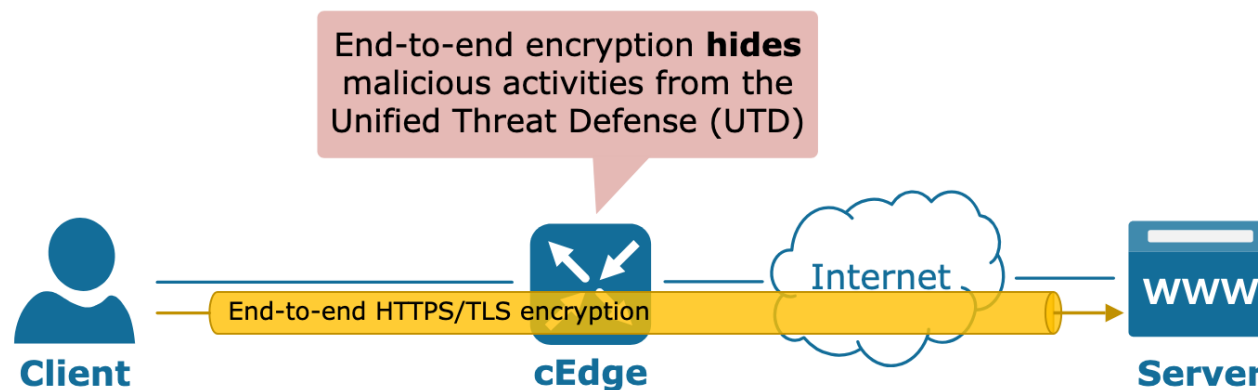
HTTPS

TLS-Terminierung beim Reverse Proxy (NGINX)



<https://www.howtoforge.com/tutorial/ubuntu-load-balancer-haproxy/>

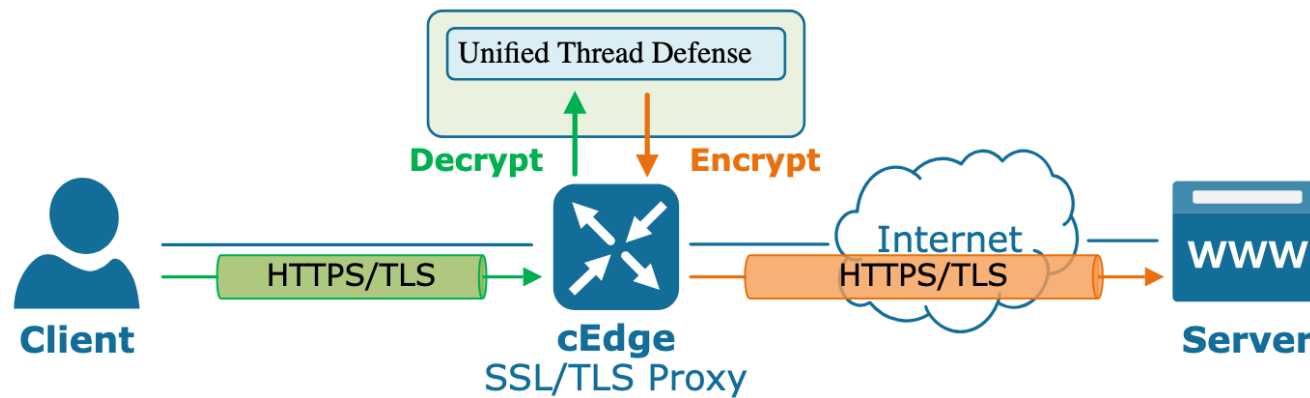
HTTPS Forward Proxy



Basis ist das HTTP-Header Feld „Connect“.

Faktisch leitet der Proxy die Nachrichten des Handshakes durch. Dabei fließen notwendige Informationen zur Bestimmung des Session Keys aktueller Ephemeral Key Exchange Verfahren nicht vom Client/Server, der Proxy kann also den Session Key nicht ermitteln

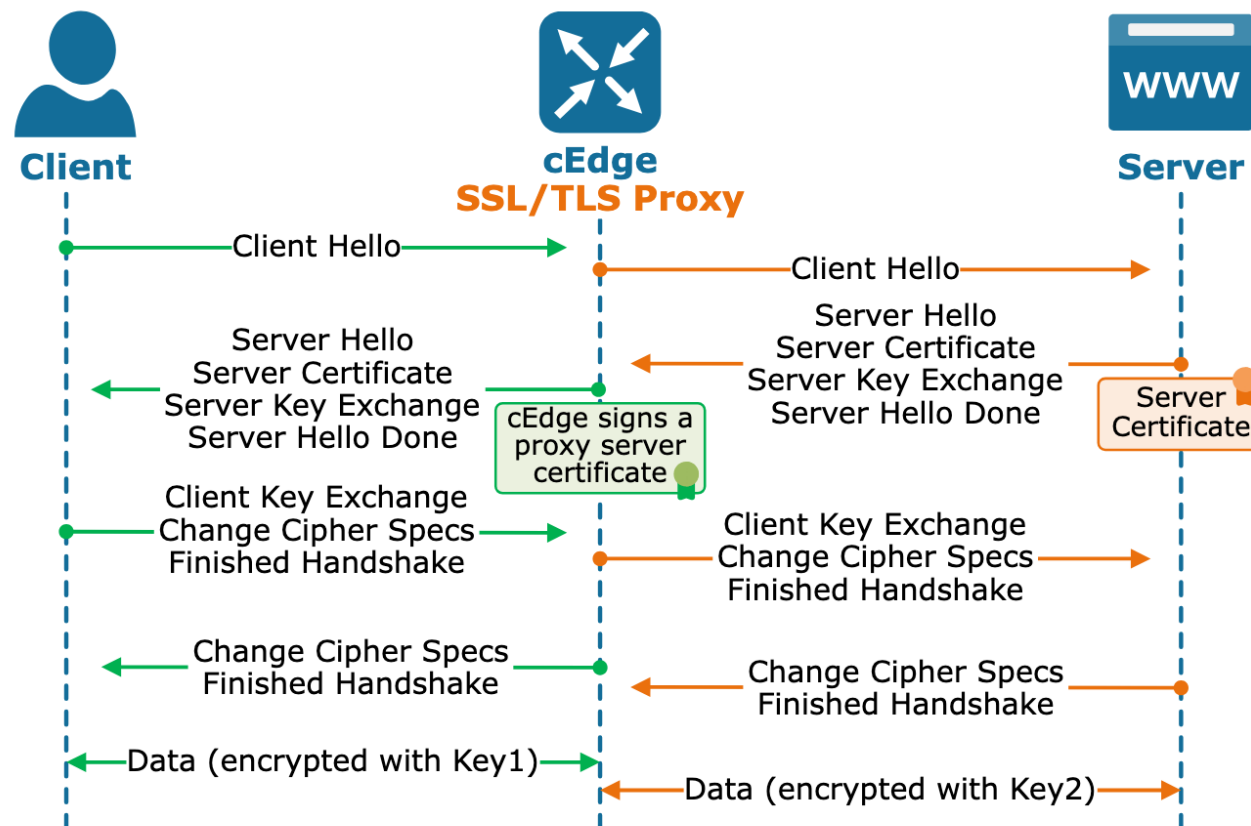
HTTPS Proxy



<https://www.networkacademy.io/ccie-enterprise/sdwan/https-tls-proxy>

HTTPS

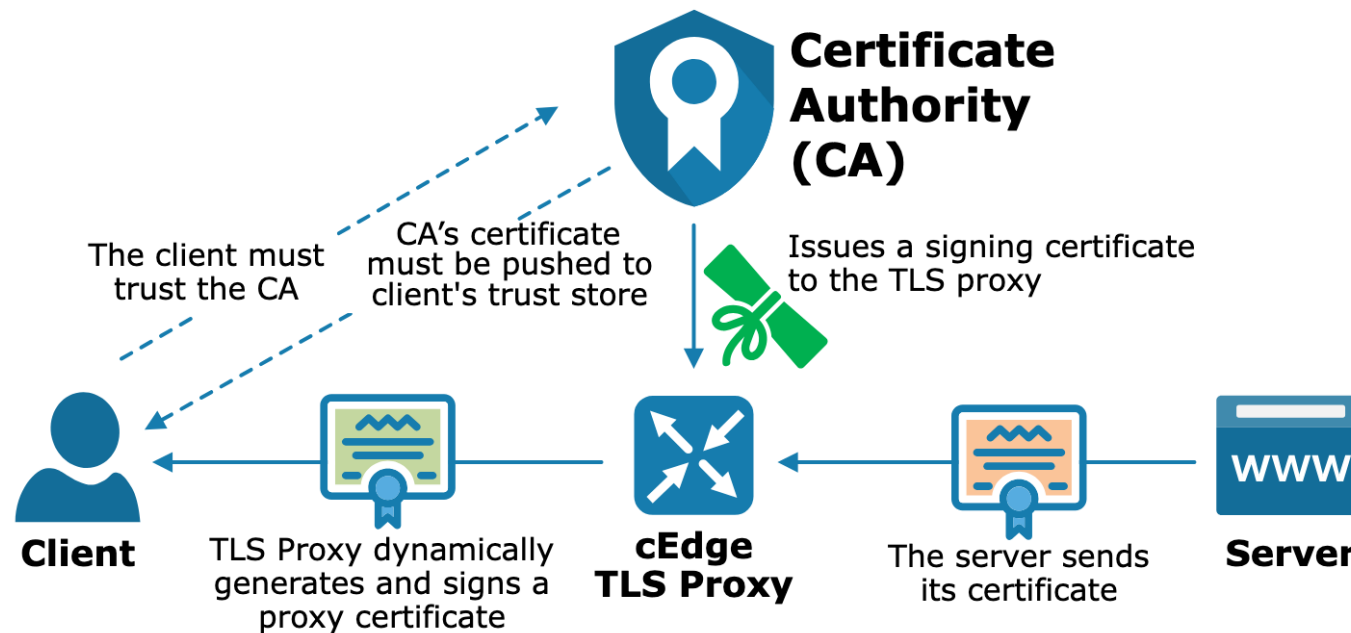
TLS/SSL Visibility Appliance



<https://www.networkacademy.io/ccie-enterprise/sdwan/https-tls-proxy>

HTTPS

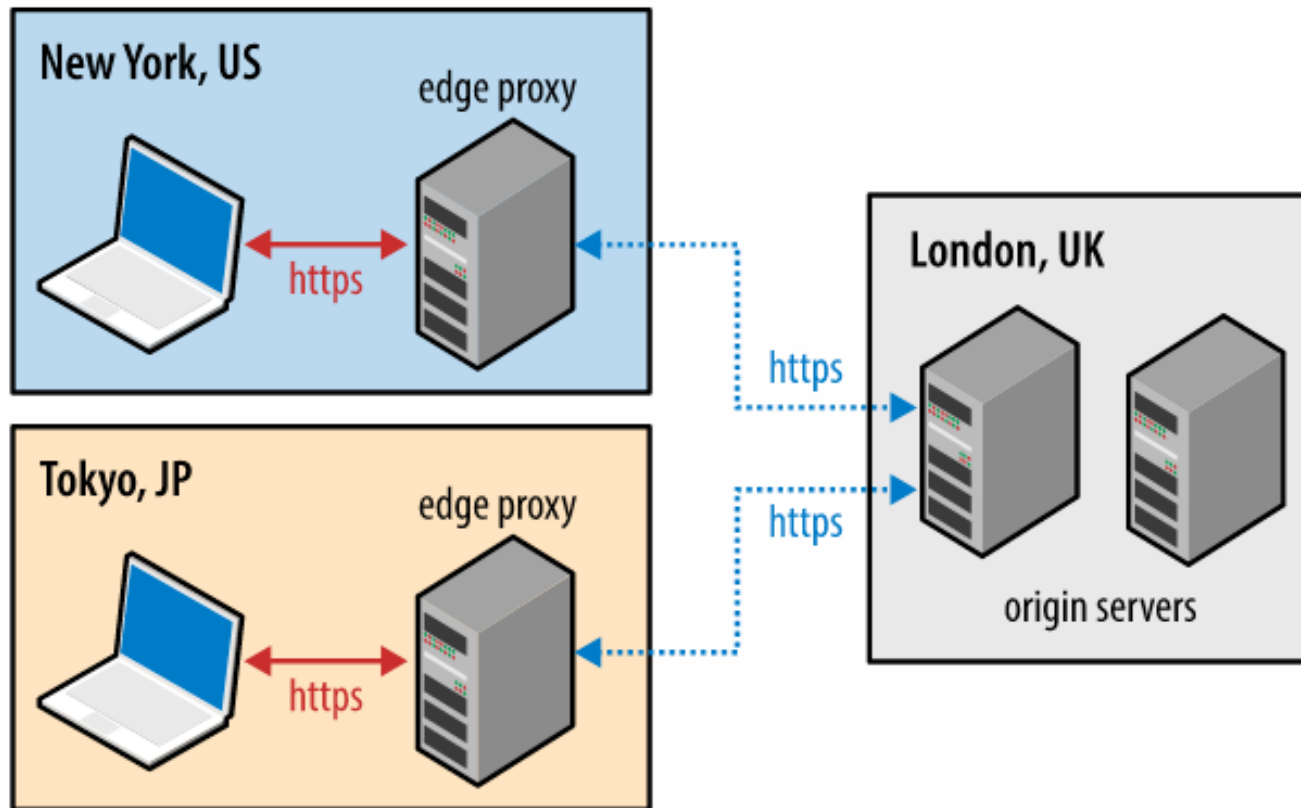
TLS/SSL Visibility Appliance



<https://www.networkacademy.io/ccie-enterprise/sdwan/https-tls-proxy>

HTTPS

TLS Early Termination – TLS/SSL Visibility Appliance



<https://hpbn.co/transport-layer-security-tls/#tls-handshake>